Greedy coverage-driven factor bases: relation yield at equal unknowns

Study `research/greedy_factor_base_yield_20261014` · run 2026-10-14 · seeds 1–20 · `koblitz_greedy_factor_base_yield` · pair sums, m = 2

1. Question and answer

Question. Can a factor base for the Koblitz index calculus be chosen so that its pair sums cover more prime-order subgroup targets than a factor base of the same size chosen at random? The count that matters is the number of columns, meaning signed Frobenius orbits, because the expected-trial formula charges one unknown per column:

$$E = (U + 1)\frac{T}{\text{covered}}, \text{ where } U \text{ is the number of columns and } T \text{ the number of nonzero targets.}$$

Answer, for the measured curves. Yes, when coverage is only partial. At equal columns and equal points, a greedy choice covers up to **18% more** targets than uniformly random orbits, and it reaches full coverage with fewer columns. At the best column count the expected number of trials falls by **1.18× on $K_0/2^{13}$, 1.08× on $K_0/2^{15}$ and 1.11× on $K_1/2^{17}$**, that is 8–18%. The gain vanishes once coverage saturates, and the construction does not yet scale past $r \approx 2^{22}$. This is a constant-factor effect measured at small and medium degree. It is not an asymptotic improvement, and it is not a cryptographic-size factor-base builder.

2. Method

2.1. Scalar model, and why the census is exact

The targets lie in the prime-order subgroup $\langle G \rangle \cong \mathbb{Z}/r\mathbb{Z}$. Writing each point as kG turns the group law into addition modulo r. Frobenius acts as multiplication by λ, so a signed Frobenius orbit is a fixed set of scalars. The coverage of a base F is therefore the number of distinct nonzero sums a + b (mod r) with a, b in F, divided by r − 1. The greedy evaluates this without curve arithmetic:

```
coverage(F) = |{ a + b mod r : a, b in F } \ {0}| / (r - 1)
```

The production census (`PairSumTable` and `witness_list`) is run on every greedy base, and the two counts are compared. They agree on all 56 greedy bases in the run.

2.2. The construction

At each step the greedy scores candidate orbits by the number of targets they newly cover as sums of two base points, including sums with each other, and adds the best one. Orbits are chosen as units, one column each. The run uses two selection rules and one control:

- **greedy**: every remaining orbit is scored at each step; deterministic.
- **greedy, 64 sampled**: a seeded sample of 64 orbits is scored at each step.
- **random orbits**: U orbits drawn uniformly from the same orbit list, by partial Fisher–Yates. Same granularity, same count, no coverage information. This is the control.

2.3. Why columns, not points

The shipped sampler (`build_subgroup_orbit_factor_base_with_cost`) is specified in points, but its batches add eight orbits at a time. On $K_0/2^{13}$ every requested budget from 22 to 179 points returns a base of 182 or 208 points (7 or 8 columns, depending on the seed), so its smallest base is one batch of about eight orbits. Comparing bases of equal points therefore compares unequal unknowns. The comparison here is at equal columns. The run also checks that greedy and random bases with the same column count have the same number of points in every matched pair (2240 pairs, 0 mismatches; each column carries exactly 2n points). The pair-table size, which the pair-table oracle pays for, is therefore matched too.

2.4. Data and controls

- Curves: $K_0/2^9$, $K_1/2^9$, $K_0/2^{13}$, $K_0/2^{15}$, $K_1/2^{15}$ and $K_1/2^{17}$. $K_0/2^{17}$ and $K_1/2^{13}$ are not valid Koblitz degrees in this code.
- Targets: exhaustive $\langle G \rangle \setminus \{O\}$, so coverage has no sampling error. The only randomness is in the random and pooled arms, over 20 seeds.
- Column counts swept: $U \in \{1, 2, 3, 4, 5, 6, 8, 10, 12, 16, 20, 24, 32, 40, 48\}$, capped by the orbits available.
- Reference: the shipped sampler, at its own budgets, recorded but not matched.
- Intervals: 95% percentile bootstrap over the 20 random seeds (4000 resamples), computed in the example.

3. Results

3.1. Expected trials at the best column count

curve	r	T	greedy U*	greedy E	random U*	random E (mean)	E ratio, random / greedy
$K_0/2^{13}$	2003	2002	3	4.46	3	5.29	1.18
$K_0/2^{15}$	751	750	2	3.00	2	3.25	1.08
$K_1/2^{17}$	65587	65586	12	16.35	12	18.18	1.11

Table 1: Expected trials E = (U + 1) T / covered at the column count U* that minimises E within each arm. Greedy is one deterministic base per U; random is the mean of 20 seeds. Source: `data/rows.json`, minimised over the swept U.

The smaller curves saturate quickly. On $K_0/2^{15}$ two greedy orbits cover all 750 targets, while two random orbits do so in only 3 of 20 seeds (three always do). On $K_0/2^9$ two orbits cover everything under both rules. The $K_1/2^9$ and $K_1/2^{15}$ subgroups are covered by a single orbit under both rules. These curves do not discriminate, so they are omitted from the figures.

3.2. Coverage at equal columns

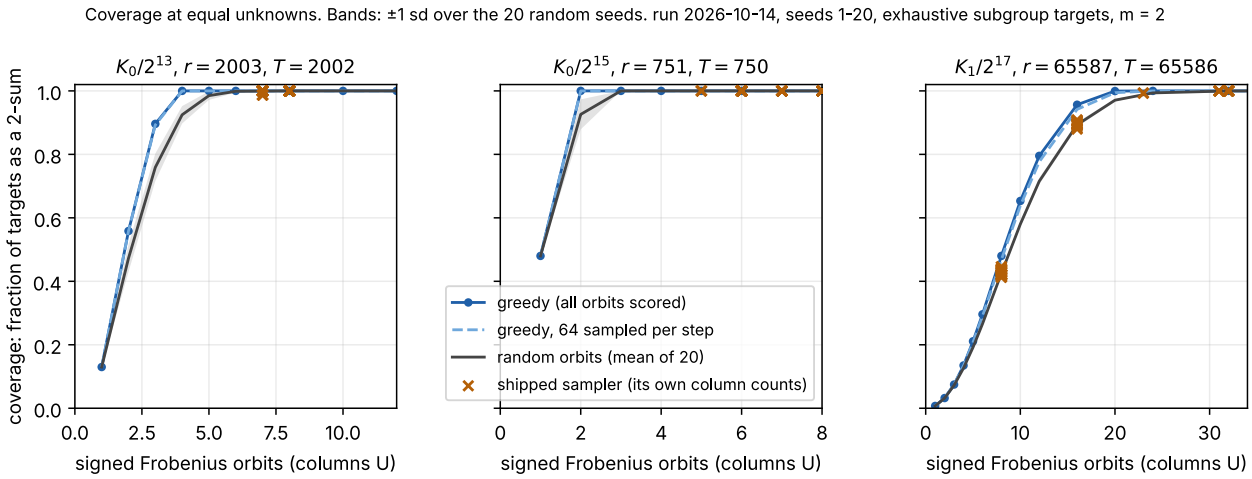


Figure 1: Coverage (fraction of T targets written as a sum of two base points) against the column count U, at equal unknowns. Solid: greedy. Dashed: greedy with 64 sampled orbits per step. Grey: uniform random orbits, mean of 20 seeds, band ±1 sd. Orange crosses: the shipped sampler at its own column counts. Run 2026-10-14; m = 2; exhaustive targets. Source: `data/summary.json`, `data/rows.json`; plot `graphs/plot_yield.py`.

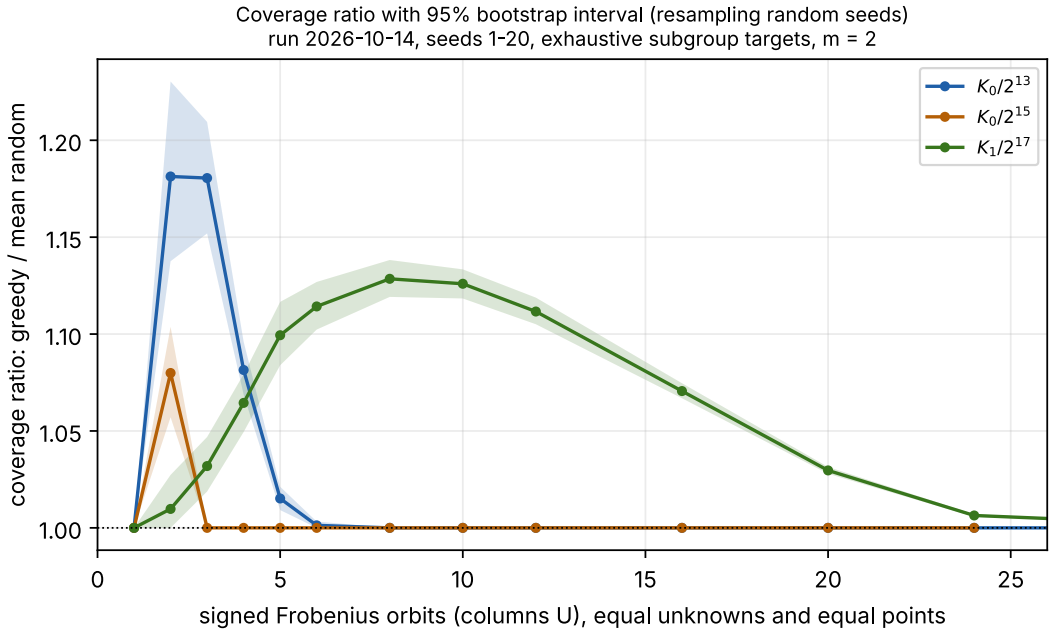


Figure 2: Coverage ratio of greedy over mean random at equal U, with a 95% bootstrap interval resampling the random seeds. The greedy side is one deterministic base per U. Source: `data/summary.json`, fields `coverage_ratio` and `coverage_ratio_ci95`.

The effect has a clear shape. On $K_0/2^{13}$ it is largest at U = 2 and U = 3, where the greedy base reaches coverage 0.558 and 0.896, against 0.473 ± 0.043 and 0.759 ± 0.044 for random orbits. The ratios are 1.181 [1.138, 1.230] and 1.180 [1.152, 1.209]. At U = 4 the greedy base is complete (1.000), while random orbits average 0.925 ± 0.028. On $K_1/2^{17}$, where r = 65587 is large enough that the sums stay partial for many columns, the ratio rises from 1.010 at U = 2 to a peak of 1.129 [1.119, 1.138] at U = 8, and falls back to 1.030 at U = 20 as both arms approach full coverage.

U	points	greedy cov.	random cov.	pooled cov.	ratio (95% CI)	E greedy / random
2	68	0.0321	0.0318 ± 0.0011	0.0321	1.010 (1.000, 1.027)	93.3 / 94.4
4	136	0.1348	0.1266 ± 0.0043	0.1343	1.064 (1.049, 1.079)	37.1 / 39.5
6	204	0.2960	0.2657 ± 0.0067	0.2901	1.114 (1.102, 1.127)	23.7 / 26.4
8	272	0.4800	0.4254 ± 0.0085	0.4683	1.129 (1.119, 1.138)	18.8 / 21.2
10	340	0.6532	0.5801 ± 0.0090	0.6383	1.126 (1.118, 1.133)	16.8 / 19.0
12	408	0.7952	0.7154 ± 0.0102	0.7767	1.112 (1.105, 1.119)	**16.3** / 18.2
16	544	0.9565	0.8934 ± 0.0076	0.9418	1.071 (1.067, 1.075)	17.8 / 19.0
20	680	0.9995	0.9707 ± 0.0039	0.9943	1.030 (1.028, 1.031)	21.0 / 21.6

Table 2: $K_1/2^{17}$ at selected column counts. Points are identical in both arms in every matched pair. Covered fractions of T = 65586. E is the census's expected trials. The bracketed interval is a 95% bootstrap over the random seeds. Source: `data/summary.json`, `data/rows.json`.

3.3. Expected trials

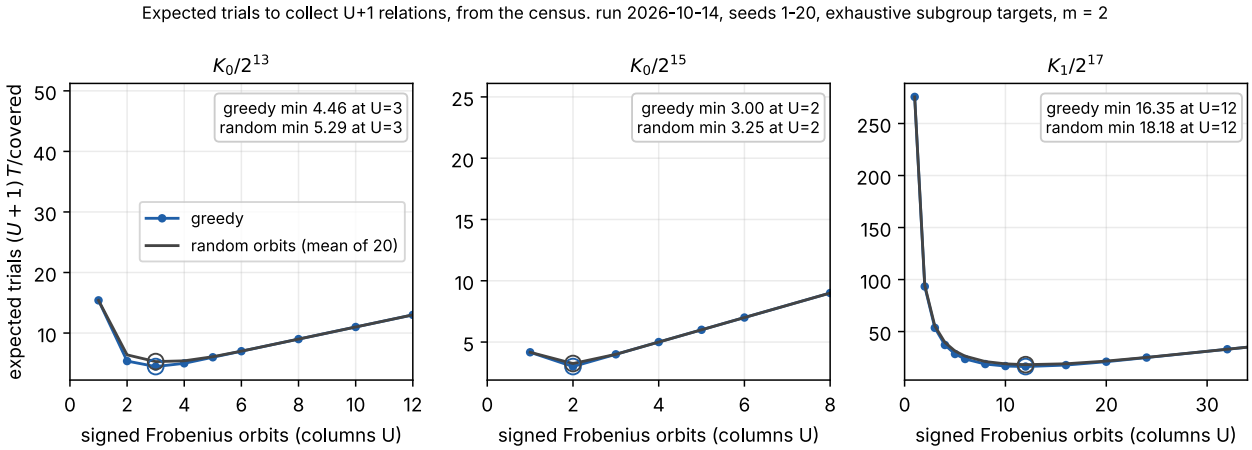


Figure 3: Expected trials E = (U + 1) T / covered against U for greedy and for the mean of 20 random seeds. Labels mark each arm's minimum. Source: `data/rows.json`, field `expected_trials`.

Figure 3 is the quantity a relation collection pays for. The greedy and random minima fall at the same column count on all three discriminating curves, and the greedy minimum is lower by the ratios in the first table.

3.4. The shipped sampler, at matched size

base	points	columns	coverage	E	construction time
shipped sampler, $K_1/2^{17}$, B = 128 or 256	272	8	0.427	21.07	0.2 ms, no enumeration
random orbits, $K_1/2^{17}$, U = 8	272	8	0.425	21.17	84 ms (orbit enumeration)
greedy, $K_1/2^{17}$, U = 8	272	8	**0.480**	**18.75**	601 ms (full scan)
shipped sampler, $K_0/2^{13}$, B = 22 to 179	182 or 208	7 or 8	0.999	8.61	0.1 ms
greedy, $K_0/2^{13}$, U = 4	104	4	**1.000**	**5.00**	5.1 ms (full scan)

At matched size the shipped sampler is indistinguishable from the random control, which is what it should be, since it draws abscissae without coverage information. The gain is the difference between choosing orbits and drawing them. On $K_0/2^{13}$ the shipped sampler's smallest base has seven or eight columns and covers 0.999 of the targets on average. A four-column greedy base already covers all of them, at about half the unknowns.

3.5. Construction cost

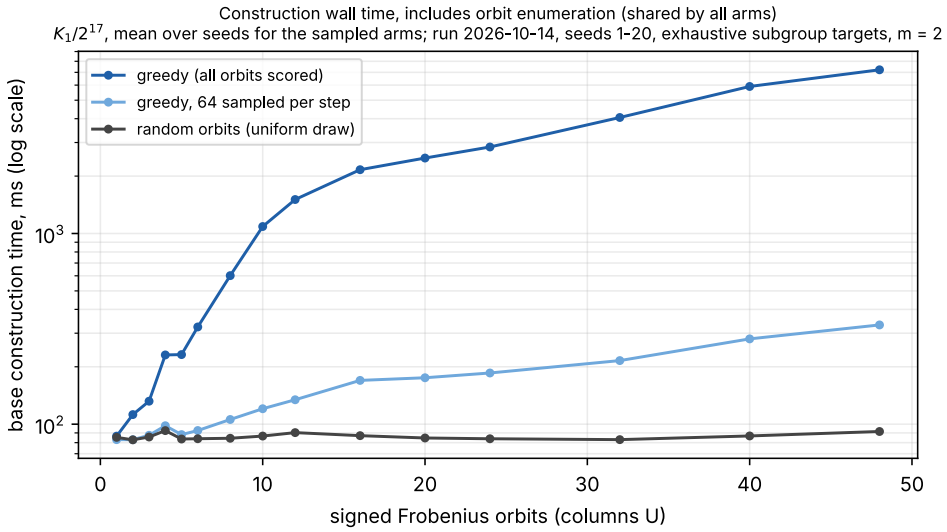


Figure 4: Construction wall time on $K_1/2^{17}$ against U. Random and greedy both include the full enumeration of ⟨G⟩ into orbits, about 85 ms, which is the whole cost of the random arm. The greedy adds the scoring. Source: `data/rows.json`, field `build_ms`. Host: 4 vCPU Intel Xeon 2.10 GHz, shared container, one run, no isolation.

The greedy scoring is the cost. On $K_1/2^{17}$ a full-scan base costs 231 ms at U = 4, 601 ms at U = 8 and 1507 ms at U = 12. Sampling 64 candidates per step keeps the cost near the enumeration floor (134 ms at U = 12) and keeps most of the gain: at U = 12 the pooled coverage ratio is 1.086, against 1.112 for the full scan. The construction is a one-off per curve and per factor base, which relation collections on the same base amortise. The shipped sampler's 0.2 ms is not comparable, because it does not enumerate the subgroup.

4. Checks

- **Scalar model against census.** For all 56 greedy bases the builder's count equals the production census (`model_match` true in every row). The unit tests check this on $K_0/2^9$ as well.
- **Matched points.** 2240 pairs of greedy or pooled and random bases with equal column count have equal point counts. The number of points per column is 2n for every curve in the run.
- **Determinism.** The full-scan greedy is independent of the seed (unit test). The pooled greedy and the random control are reproducible from their seeds.
- **Prefix stability.** The greedy base with k columns is the first k selected orbits of the base with more columns, so one run gives the whole coverage trajectory (`coverage_by_column`).
- **Serialisation.** The `greedy_coverage` recipe round-trips through JSON and materialises to the direct build (unit test), so the `ic` tool can read it as a factor-base recipe.
- **Regression.** The existing factor-base search tests pass (11 of 11), and the new module has 7 passing unit tests.

5. Limits

1. **Scale.** The construction enumerates ⟨G⟩ to form orbits and keeps a coverage array of length r. The implementation is capped at $r \le 2^{22}$ (`MAX_SUBGROUP_ORDER`). It cannot build a base for a cryptographic-size curve as written. Scaling it needs a candidate generator from sampled abscissae, and a hashed set of covered sums in place of a dense array. That is a follow-up, not something measured here.
2. **Degrees.** Only $n \in \{9, 13, 15, 17\}$ were measured. $K_1/2^9$ and $K_1/2^{15}$ are saturated by one orbit and do not discriminate. Degree 19 was not run: the full scan is too slow at that size, and the pooled arm is the candidate for it.
3. **Only pair sums.** m = 2. The effect at m = 3 and with the algebraic oracles is not measured. The coverage counted here is the pair-table oracle's.
4. **Trials, not solve time.** The metric is the census's expected trials, (U + 1) T / covered. Per-trial oracle cost is not measured. Because the bases match in points, the pair-table cost per trial is matched; this is an inference, not a measurement.
5. **Heuristic.** The greedy maximises the gain of the next orbit and does not look ahead. No optimality is claimed. It is deterministic only in the full-scan form.
6. **Variance.** The greedy arm is one deterministic base per (curve, U). Its interval reflects the random arm's seed variance only.
7. **Saturation.** Once coverage is complete, the two rules tie by construction. The gains are confined to the partial-coverage regime and the first few columns.

6. Conclusion

Within these limits, the greedy coverage rule is a real improvement over choosing orbits at random. It is worth 8–18% of expected relation trials at the best column count on the measured curves. It does not change the asymptotic cost, and as

written it works only at small and medium degree. The shipped sampler performs like the random control at the same size. Making the greedy usable at cryptographic size is the next step, and it is not done here.

7. Reproduction

```
# run (about 10 minutes, release build, 4 cores)
cargo run --release --example koblitz_greedy_factor_base_yield -- \
    research/greedy_factor_base_yield_20261014/data 9,13,15,17 20
# figures from the recorded values
python3 research/greedy_factor_base_yield_20261014/graphs/plot_yield.py
```

Files: `data/rows.json` (one row per base, 2776 rows), `data/summary.json` (56 column counts), `data/run.log` (the run's stderr), `graphs/plot_yield.py` (the figure source), `graphs/fig*.svg` (vectors) and `graphs/fig*.png` (150 dpi renders).